Is Reinforcement Learning Cheaper than LQR?

Fuel-Efficient Station-Keeping at the Unstable Sun–Earth L_1 Point

Emir Eymen Sargın
TEV İnanç Lisesi, Kocaeli, Türkiye

Draft, October 28, 2026

Abstract

A spacecraft at the Sun–Earth L_1 point drifts away exponentially, with an e-folding time of 23.0 days, unless it fires its thrusters regularly. We ask whether a deep reinforcement learning (RL) agent can hold it there with less fuel than a linear–quadratic regulator (LQR). All controllers fly the same nonlinear planar circular restricted three-body model with navigation errors, thrust errors and unmodelled forces, and are scored on 20 held-out random seeds. A DQN agent with 25 discrete actions keeps the spacecraft in every test episode but needs 2–12× the fuel of LQR. With a continuous action and a cubic thrust map, PPO reaches the right thrust scale and spends 0.110 ± 0.007 m/s per year while holding (LQR: 0.062 m/s). Comparing against LQR tuned to the *same* tracking accuracy, PPO needs 1.3–2.7× LQR's fuel across three training seeds and never reaches the LQR fuel–accuracy curve. Most of the DQN–LQR gap therefore came from the action representation, not from learning, but LQR remains Pareto-optimal. Two single-run findings did not survive retraining with new seeds, including an apparent advantage of PPO under a thrust limit; we report them as a cautionary example. Code and data are open source.

1 Introduction

The collinear Lagrange points L_1 and L_2 of the Sun–Earth system are attractive places for observatories: SOHO watches the Sun from L_1, and JWST observes from near L_2. Both points are equilibria of the circular restricted three-body problem (CR3BP), but unstable ones [7, 15]. A spacecraft left exactly at L_1 with a 150 m error drifts away by a factor e every 23.0 days and is more than a million kilometres away within a year (Fig. 1). Missions therefore perform *station-keeping*: small, regular thruster firings that cancel the unstable motion [3, 6].

Classical station-keeping uses linear control around the reference. Recently, deep reinforcement learning has been applied to guidance and station-keeping in multi-body dynamics [8, 9, 11, 14], often with promising results. Our question is narrower and deliberately simple:

> *On the most basic station-keeping problem, holding the L_1 point itself, can a learned controller beat LQR on fuel at the same tracking accuracy?*

We expected LQR to be hard to beat, because the motion stays in a region where the linearised dynamics are very accurate and LQR is the optimal controller for linear dynamics and quadratic cost. The value of the experiment is in measuring *how close* RL gets, and *why* it falls short.

Our contributions are:

1. A from-scratch CR3BP simulator with a shared station-keeping environment, so that LQR and RL see identical physics, errors and fuel accounting, and a fixed test protocol on seeds never used in training or model selection.

2. A fair comparison: each learned controller is compared with LQR tuned to the same tracking accuracy, interpolated on a 57-point LQR sweep.

Table 1: Sun–Earth Lagrange points computed by the simulator. λ is the largest real part of the eigenvalues of A, in nondimensional units; the e-folding time is $1/\lambda$ in days.

Point	Distance from Earth	Unstable eigenvalue λ	e-folding time
L1	1,491,539 km	2.533	23.0 days
L2	1,501,519 km	2.484	23.4 days
L3	299,195,479 km	0.0028	57 years
L4, L5	1 AU (60° ahead/behind)	0	stable (Routh: $\mu < 0.0385$)

3. Evidence that the action representation, not the learning algorithm, is the main reason DQN needs much more fuel: a continuous PPO agent with a cubic thrust map reduces the gap from 2–12× to 1.3–2.7×.

4. A seed study showing which conclusions hold across training seeds and which do not [1, 5].

2 Model

2.1 Circular restricted three-body problem

We use the standard nondimensional units: the Sun–Earth distance is 1, the total mass is 1 and the Earth's orbital period is 2π, so one time unit is 58.13 days. The mass ratio is $\mu = m_E/(m_S + m_E) = 3.0034 \times 10^{-6}$. In the frame rotating with the Earth, with the Sun at $(-\mu, 0)$ and the Earth at $(1-\mu, 0)$, the planar equations of motion with a control acceleration $\mathbf{u} = (u_x, u_y)$ are

$$\ddot{x} - 2\dot{y} = \frac{\partial \Omega}{\partial x} + u_x, \qquad \ddot{y} + 2\dot{x} = \frac{\partial \Omega}{\partial y} + u_y, \qquad \Omega = \frac{x^2 + y^2}{2} + \frac{1-\mu}{r_1} + \frac{\mu}{r_2}, \tag{1}$$

where r_1 and r_2 are the distances to the Sun and the Earth. With $\mathbf{u} = 0$ the Jacobi constant $C = 2\Omega - \dot{x}^2 - \dot{y}^2$ is conserved, which we use to validate the integrators. Over ten years, the adaptive DOP853 integrator [4] keeps the relative error in C below 10^{-13}, and the fixed-step RK4 integrator used inside the simulator converges with a measured order of 3.92 (theory: 4).

2.2 Lagrange points and linear stability

The collinear points are the roots of $\partial\Omega/\partial x = 0$ on the x axis. Linearising Eq. (1) about a point gives $\dot{\mathbf{x}} = A\mathbf{x}$ with $\mathbf{x} = (\delta x, \delta y, \delta\dot{x}, \delta\dot{y})$. At L_1 the eigenvalues of A are a saddle pair $\pm\lambda$ and a centre pair $\pm i\omega$, with $\lambda = 2.533$ (Table 1). The saddle makes L_1 unstable: a small error grows like $e^{\lambda t}$. Without control, a spacecraft that starts 1000 km from L_1 is more than 10^4 km away after 61 days, and even one that starts exactly at L_1 is pushed out by the unmodelled forces of Section 3 within 222–314 days.

3 The station-keeping problem

Simulator. The spacecraft follows the full nonlinear dynamics (1). Once per day a controller reads a noisy estimate of its position and velocity relative to L_1 and commands a thrust acceleration, which is held constant for the day (zero-order hold). Fuel is measured as the total velocity change actually delivered, $\Delta v = \sum_k \|\mathbf{u}_k\|\,\Delta t$. An episode lasts one year; the spacecraft is *lost* if it is ever more than 10 000 km from L_1.

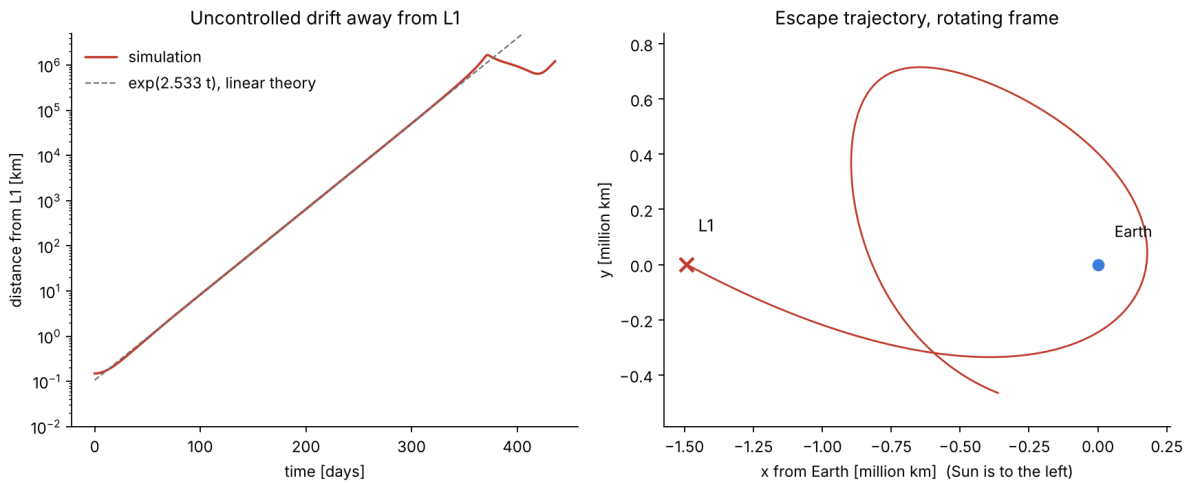


Figure 1: Uncontrolled drift from L_1, starting 150 m away at rest. Left: the distance grows at the rate predicted by the unstable eigenvalue. Right: the escape trajectory in the rotating frame.

Error model. Every controller faces the same errors (1σ, independent per axis and per day, drawn from the same random seed for every controller): navigation errors of 1 km in position and 1 mm/s in velocity; a thrust magnitude error of 2 % and a pointing error of 1°; and an unmodelled acceleration of 10^{-9} m/s^2, which stands in for changes in solar radiation pressure and the pull of the Moon and planets. These values are illustrative assumptions for a deep-space mission, not those of a specific spacecraft.

Scenarios and metrics. *Holding*: the spacecraft starts at L_1 at rest. *Recovery*: it starts 1000 km away on the Sun–Earth line. For each episode we record Δv over the year, the mean and RMS distance from L_1, and whether the spacecraft was kept for the full year.

Test protocol. During training and model selection, agents are only ever evaluated on seeds 0–4. All results in this paper use seeds 100–119, which are never touched until the final evaluation. Because the seed fixes every random draw, all controllers meet exactly the same navigation errors, thrust errors and disturbances, draw for draw.

Simplifications. Real L_1 missions fly halo or Lissajous orbits around the point rather than holding the point itself, the motion is three-dimensional, and manoeuvres are usually impulsive and weeks apart. We keep the problem planar, hold the point, and use a small daily thrust so that the comparison between controllers is clean; Section 6 discusses what this means for the conclusions.

4 Controllers

4.1 LQR baseline

Because the thrust is constant over each day, the linear model can be discretised exactly with a zero-order hold, $\mathbf{x}_{k+1} = A_d\mathbf{x}_k + B_d\mathbf{u}_k$, where A_d and B_d come from one matrix exponential. The LQR gain K minimises $\sum_k \mathbf{x}_k^\top Q\mathbf{x}_k + \mathbf{u}_k^\top R\mathbf{u}_k$ and follows from the discrete algebraic Riccati equation [2]. We set the weights with Bryson's rule,

$$Q = \mathrm{diag}\left(\frac{1}{r_{\mathrm{ref}}^2}, \frac{1}{r_{\mathrm{ref}}^2}, \frac{1}{v_{\mathrm{ref}}^2}, \frac{1}{v_{\mathrm{ref}}^2}\right), \qquad R = \frac{\rho}{u_{\mathrm{ref}}^2} I_2, \tag{2}$$

Table 2: Training hyperparameters (Stable-Baselines3).

DQN (run 2)		PPO	
Steps	10^6	Steps	3×10^6 (8 envs)
Learning rate	10^{-4}	Learning rate	3×10^{-4}
Replay buffer	2×10^5	Rollout per env	512
Batch size	256	Batch size / epochs	256 / 10
Target update	every 10^4 steps	GAE λ / clip	0.95 / 0.2
Exploration	ε: $1 \to 0.02$ over 30 %	Initial log std	-1
Network	128–128	Networks (policy, value)	128–128 each
γ	0.99	γ	0.99

with $r_{\text{ref}} = 1000$ km, $v_{\text{ref}} = 1$ m/s and $u_{\text{ref}} = 10^{-6}$ m/s^2, and vary the single fuel weight ρ: a larger ρ means less thrust and looser tracking. The controller is $\mathbf{u}_k = -K\hat{\mathbf{x}}_k$, applied directly to the noisy estimate $\hat{\mathbf{x}}_k$ without a filter. As a reference we use $\rho = 3$, the cheapest recovery in an initial sweep.

4.2 Reinforcement learning environment

The RL agents fly the same simulator through a Gymnasium interface. The observation has 8 values computed from the noisy estimate: position in units of 1000 km, velocity in m/s, and the symmetric logarithm $\operatorname{sign}(z)\log_{10}(1+|z|)$ of the position in km and the velocity in mm/s, so that both 0.1 km and 1000 km are visible to the network. The reward per day is

$$r_k = 1 - w_{\text{pos}} \log_{10}\!\left(1 + \frac{d_k}{1\ \text{km}}\right) - w_{\text{fuel}}\,\frac{\Delta v_k}{1\ \text{m/s}} - 100 \cdot \mathbb{1}[\text{lost}], \tag{3}$$

with d_k the distance from L_1, $w_{\text{pos}} = 0.3$ and $w_{\text{fuel}} = 10$. Training episodes start at a random offset of up to 1000 km with a small random velocity, so both holding and recovery are learned.

DQN (discrete thrust). DQN [10] chooses each day one of 25 actions: coast, or thrust in one of 8 directions at 10^{-8}, 10^{-7} or 10^{-6} m/s^2. We trained two runs of 1M steps; run 2 uses a lower learning rate, slower target updates and a stronger distance penalty after run 1 became unstable late in training.

PPO (continuous thrust). PPO [13] outputs a two-dimensional action $\mathbf{a}$ with $\|\mathbf{a}\| \le 1$, which is mapped to thrust by

$$\mathbf{u} = u_{\max}\,\|\mathbf{a}\|^{p-1}\,\mathbf{a}, \qquad u_{\max} = 10^{-6}\ \text{m/s}^2, \tag{4}$$

the same limit as DQN's largest action. With the plain linear map ($p = 1$), the holding thrust LQR uses (about 2×10^{-9} m/s^2) corresponds to an action of norm 2×10^{-3}, a value the Gaussian policy struggles to output precisely. With the cubic map ($p = 3$) the same thrust needs $\|\mathbf{a}\| \approx 0.12$, while the full thrust is still available. We train both maps to isolate this choice.

Training and model selection. Both algorithms use the Stable-Baselines3 implementations [12] with the hyperparameters in Table 2. Every 25 000 (DQN) or 50 000 (PPO) steps, the deterministic policy flies holding and recovery on seeds 0–4 and is scored by $\frac{1}{2}\sum_{\text{scen}}\big(100\,\frac{\text{days kept}}{365} - 10\,\Delta v - \log_{10}(1+\bar{d})\big)$; the best checkpoint of each run is kept. Each PPO configuration was trained with three training seeds (0, 1, 2) and otherwise identical settings, 3M steps each with 8 parallel environments (about 11 minutes on one CPU core per run).

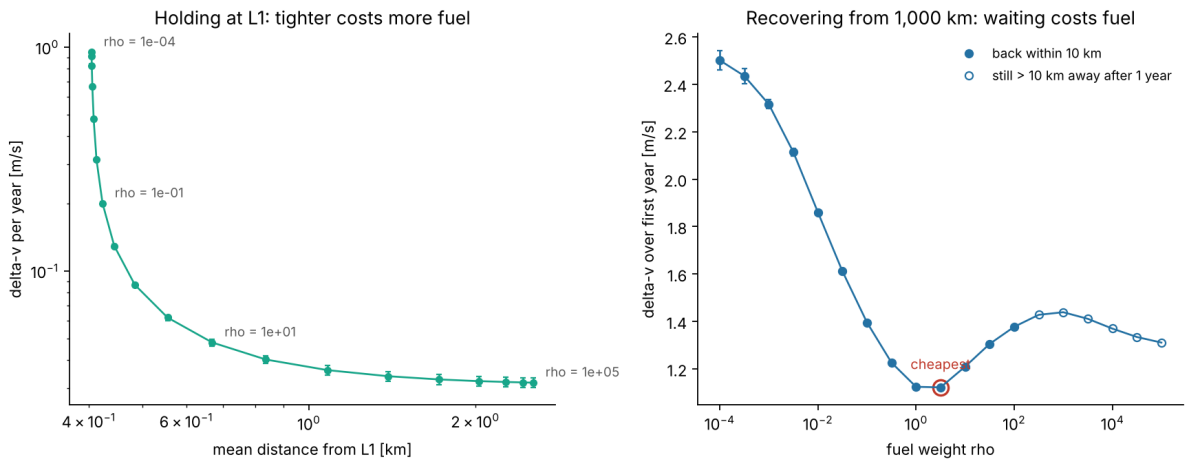


Figure 2: LQR fuel versus tracking accuracy as the fuel weight ρ is varied (5 seeds per point). Left: holding. Right: recovery from 1000 km.

Table 3: Test results (seeds 100–119). PPO rows are mean ± standard deviation over three training seeds; "Kept" is the share of episodes kept for the full year.

	Holding (start at L1)		Recovery (start 1000 km)		
Controller	Δv [m/s]	mean / RMS dist. [km]	Δv [m/s]	mean dist. [km]	Kept
No control	0	lost after 250 d	0	lost after 61 d	0 %
LQR, $\rho = 3$	0.062	0.52 / 0.59	1.12	76	100 %
DQN (run 2, seed 0)	0.723	1.41 / 1.62	2.17	68	100 %
PPO, linear map ($p = 1$)	0.393 ± 0.039	0.45 ± 0.04 / 0.51 ± 0.04	1.72 ± 0.12	72 ± 14	100 %
PPO, cubic map ($p = 3$)	0.110 ± 0.007	0.57 ± 0.17 / 0.64 ± 0.18	1.46 ± 0.18	84 ± 30	100 %

5 Results

5.1 The LQR fuel–accuracy trade-off

Sweeping ρ over 9 orders of magnitude (Fig. 2) shows the trade-off the agents must beat. While holding, LQR spends 0.03–0.95 m/s per year for a mean distance of 2.5–0.4 km. Below about 0.4 km the navigation error sets a floor and extra fuel buys almost nothing. For recovery the curve is not monotonic: the cheapest gain ($\rho \approx 3.2$, 1.12 m/s) sits in the middle, and *lazier* gains cost *more*, because the unstable mode keeps growing while the controller waits.

5.2 Main comparison

Table 3 summarises the test results. Every learned controller keeps the spacecraft in every test episode, so the question is purely one of fuel and accuracy. DQN needs 2–12× the fuel of LQR. PPO with the cubic map spends 0.110 ± 0.007 m/s while holding, much closer to LQR's 0.062 m/s, and the linear map lies in between.

5.3 Fair comparison at the same accuracy

The agents do not track at exactly the accuracy of LQR with $\rho = 3$, and fuel can always be traded for accuracy. We therefore compare each model with LQR tuned to the same mean distance. LQR is swept over 57 values of ρ between 10^{-3} and 10^{4} on the test seeds. Along the sweep, tighter tracking costs more fuel only up to the Δv minimum; beyond it, lazier gains cost more again, so only this left branch is a sensible reference. On it we interpolate (log–log) the LQR fuel at the model's mean distance and report the ratio. If a model is *less* accurate than the

Table 4: Fuel of each PPO model divided by the fuel of LQR tuned to the same mean distance (test seeds). "Best checkpoint" is the training step of the selected model.

Thrust map	Training seed	Holding	Recovery	Best checkpoint
cubic	0	1.54	1.44	250k
cubic	1	1.31	1.13	250k
cubic	2	2.74	dominated (≥ 1.37)	200k
linear	0	4.48	1.61	2.95M
linear	1	5.92	dominated (≥ 1.48)	1.20M
linear	2	2.90	dominated (≥ 1.50)	2.90M
cubic	mean ± std	1.87 ± 0.77	–	
linear	mean ± std	4.43 ± 1.51	–	

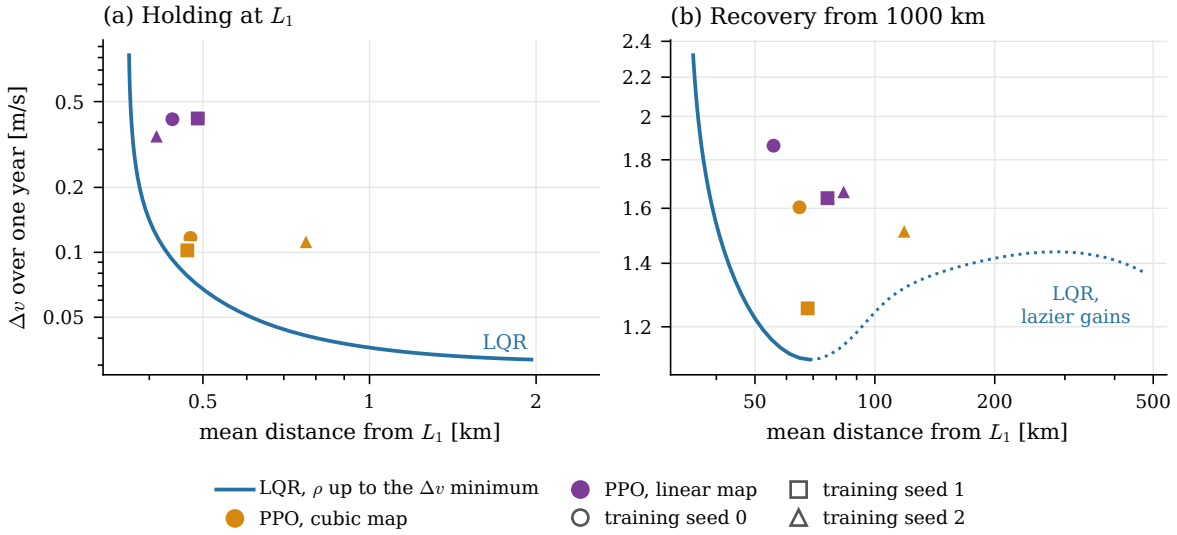


Figure 3: PPO models from three training seeds (markers) against the LQR trade-off on the test seeds (solid: ρ up to the Δv minimum; dotted: lazier gains, which cost more fuel in recovery). Every model lies above the LQR curve.

cheapest LQR ($\rho \approx 1.8$: 1.11 m/s at 69 km in recovery), that LQR is both cheaper and more accurate; we call the model *dominated* and give the ratio to the cheapest LQR as a lower bound.

Table 4 and Fig. 3 show the result. While holding, the cubic-map agent needs 1.3–2.7× LQR's fuel at the same accuracy (mean 1.87 ± 0.77), the linear map 4.43 ± 1.51. In recovery, the cubic agent needs 1.1–1.4× or is dominated. **No model, in any training seed or scenario, reaches the LQR curve: LQR is Pareto-optimal on this problem.**

5.4 Why the gap shrank: the action representation

The thrust logs explain the difference between the agents (holding, test seed 100). LQR's median daily thrust is 1.79×10^{-9} m/s^2, far below DQN's smallest non-zero action of 10^{-8} m/s^2. DQN run 1 therefore fires that smallest level on 351 of 365 days, and run 2 coasts on 214 days but over-corrects with 10^{-7} m/s^2 on 97. The cubic PPO agent's median thrust is 1.73×10^{-9} m/s^2, almost exactly LQR's: it has learned the right thrust scale. Its extra fuel comes from the tail, with 27 days above 10^{-8} m/s^2 against 0 for LQR. With the linear map the median is 1.0×10^{-8} m/s^2 and 185 days exceed 10^{-8} m/s^2.

The linear and cubic configurations use the same algorithm, reward and hyperparameters and differ only in Eq. (4). Across the three training seeds the linear map needs 3.1–4.1× the cubic

Table 5: Robustness on the test seeds. Upper part: holding, Δv [m/s] / RMS distance in the second half of the year [km]; PPO entries are the range over the three training seeds. Lower part: share of recovery runs kept. DQN is run 2.

Sweep	LQR	PPO cubic (seeds 0 / 1 / 2)	DQN
All errors ×0	0.00 / 0.0	0.00–0.01 / 0.7–1.5	0.36 / 2.7
All errors ×1	0.06 / 0.6	0.10–0.12 / 0.5–0.8	0.72 / 1.6
All errors ×8	0.50 / 4.9	2.02–2.81 / 4.2–4.3	5.49 / 9.6
Unmodelled force ×100	1.13 / 32.8	1.87–2.18 / 6.1–9.6	4.10 / 9.6
Start 2,000 km: kept	100 % (sat. 100 %)	100 / 100 / 100 %	65 %
Start 3,000 km: kept	100 % (sat. 50 %)	100 / 50 / 50 %	50 %
Start 5,000 km: kept	100 % (sat. 50 %)	50 / 50 / 50 %	25 %

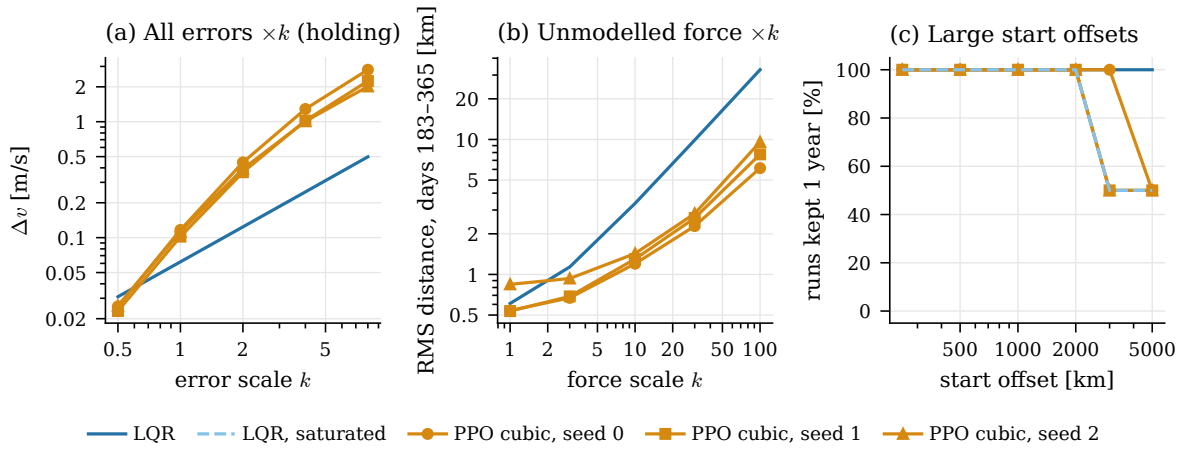


Figure 4: Robustness on the test seeds for LQR ($\rho = 3$), LQR saturated at the agents' thrust limit, and the three cubic PPO models. (a) Fuel while holding as all errors are scaled by k. (b) RMS distance in the second half of the year as only the unmodelled force is scaled. (c) Share of recovery runs kept from larger starting offsets. Fuel and distance are averaged over the runs kept.

map's holding fuel and 1.1–1.3× its recovery fuel, with no overlap between the two configurations. Changing only how small thrusts are represented therefore removes most of the distance between the learned controllers and LQR, and we conclude that **most of the DQN–LQR gap was the action representation, not the learning algorithm**. With the cubic map, the best checkpoint came early in all three seeds (200k–250k of 3M steps), and longer training never improved the selection score again; with the linear map the best checkpoints came late (1.2M–2.95M steps).

5.5 Robustness outside the training conditions

All controllers were trained or tuned with the nominal error model and offsets of at most 1000 km. We pushed them beyond this on the test seeds (Table 5, Fig. 4): all errors scaled by a factor k, the unmodelled force alone scaled by k, and starting offsets of up to 5000 km in four directions. Because the agents can thrust at most 10^{-6} m/s^2 and plain LQR has no limit, LQR is also flown saturated at that limit ("sat.").

Noise. No controller lost a run at up to 8× the nominal errors (0 runs lost in the noise and disturbance sweeps). PPO's fuel overhead grows with the noise, from 1.7–1.9× LQR at nominal levels to 4.0–5.6× at 8×: the network reacts to navigation noise, while LQR's response is linear in the error by construction, so zero-mean noise largely cancels.

Unmodelled forces. With a 100× stronger unmodelled force, LQR with $\rho = 3$ drifts much further out, because its gain was tuned for nominal conditions, while PPO holds tighter for more fuel. The two now sit at different points of the trade-off, so neither result is a win.

Large offsets and the thrust limit. Almost every lost run started on the Sun–Earth line, the direction of the unstable mode. A rough estimate of the mode's pull at distance r is $\lambda^2 r$: 7.6×10^{-7} m/s^2 at 3000 km and 1.3×10^{-6} m/s^2 at 5000 km. At 5000 km it exceeds the thrust limit, so every bounded controller loses exactly those runs, while unlimited LQR keeps all of them at a peak thrust of 4.9×10^{-6} m/s^2. At 3000 km recovery is possible but marginal: saturated LQR, whose gain ignores the limit, loses 10 runs. PPO training seed 0 keeps all of them, at a cost of about 13 m/s, and we initially reported this as an advantage of RL. **Training seeds 1 and 2 lose exactly the same runs as saturated LQR**, so this was a property of one training run, not of the method.

6 Discussion

Why LQR wins. Near L_1 the deviations stay below about a thousand kilometres, where the linearised dynamics are an excellent approximation of the true ones. For linear dynamics and a quadratic cost, LQR is the optimal state-feedback controller, and its gain is computed exactly rather than learned from noisy returns. An RL agent has to rediscover this linear law from samples, through a reward that only approximates the fuel–accuracy trade-off, and its small imperfections show up as occasional large thrusts. The setting most favourable to RL is the strongly nonlinear one, with large offsets near the thrust limit, but there our only advantage was not reproducible.

The action representation matters more than the algorithm. The cubic thrust map is a one-line change in the environment, yet it removed most of the gap between the learned controller and LQR. We think this is the most transferable lesson: when the useful actions span several orders of magnitude, the action parametrisation should make the common, small actions easy to output.

Single runs can mislead. Two of our first-run conclusions did not survive retraining: the fuel ratio ("1.6× LQR while holding" for seed 0 becomes 1.3–2.7× across seeds) and the advantage under the thrust limit, which disappeared entirely. This matches the warnings of Henderson et al. [5] and Agarwal et al. [1]. Three seeds are still few; the ranges we report should be read as the minimum variability, not a confidence interval.

Limitations. The problem is planar and holds the point itself, not a halo or Lissajous orbit, so the absolute Δv values should not be compared with real missions. Thrust is a small continuous push updated daily, rather than impulsive manoeuvres weeks apart. Neither controller uses a state estimator: a Kalman filter would help both, and would probably help PPO more, since it is the more sensitive to navigation noise. The RL reward and the LQR cost are different functions, which is why we compare on the measured fuel–accuracy plane instead. Finally, we tuned the RL hyperparameters only lightly; better tuning could narrow the gap, but the agents would still have to beat a controller that is optimal for the linearised problem.

7 Conclusion

On station-keeping at the Sun–Earth L_1 point, a classical LQR controller remains Pareto-optimal on fuel and accuracy against DQN and PPO agents trained on identical physics and errors.

Most of the initial gap between DQN and LQR came from the coarse discrete action set: with a continuous action and a cubic thrust map, PPO reaches the right thrust scale and needs 1.3–2.7× LQR's fuel at the same accuracy instead of 2–12×. Natural next steps are halo-orbit station-keeping in three dimensions, adding a navigation filter to both controllers, and hybrid designs in which a learned policy corrects a linear controller only where the linearisation fails.

Reproducibility

All code, trained models and raw results are available at `https://github.com/eeymens/l1-station-keeping` under the MIT licence. `make reproduce` regenerates every figure and table from the trained models, and every number in this paper is written into the manuscript by `scripts/11_paper_assets.py` from the raw CSV files.

Use of AI tools

[TODO: Describe accurately how AI tools were used (for example, writing parts of the simulation code and drafting this manuscript) and how you checked the results. Many venues and competitions require this statement.]

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[2] Arthur E. Bryson and Yu-Chi Ho. *Applied Optimal Control: Optimization, Estimation, and Control*. Hemisphere, 1975.

[3] Robert W. Farquhar. The control and use of libration-point satellites. Technical Report TR R-346, NASA, 1970.

[4] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, 2nd edition, 1993.

[5] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[6] Kathleen C. Howell and Henry J. Pernicka. Stationkeeping method for libration point trajectories. *Journal of Guidance, Control, and Dynamics*, 16(1):151–159, 1993.

[7] Wang Sang Koon, Martin W. Lo, Jerrold E. Marsden, and Shane D. Ross. *Dynamical Systems, the Three-Body Problem and Space Mission Design*. Marsden Books, 2011.

[8] Nicholas B. LaFarge, Daniel Miller, Kathleen C. Howell, and Richard Linares. Autonomous closed-loop guidance using reinforcement learning in a low-thrust, multi-body dynamical environment. *Acta Astronautica*, 186:1–23, 2021.

[9] Nicholas B. LaFarge, Kathleen C. Howell, and David C. Folta. Adaptive closed-loop maneuver planning for low-thrust spacecraft using reinforcement learning. In *73rd International Astronautical Congress (IAC)*, 2022.

[10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[11] Andrew B. Molnar. Hybrid station-keeping controller design leveraging Floquet mode and reinforcement learning approaches. Master's thesis, Purdue University, 2020.

[12] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[14] Takumi Suda, Yuri Shimane, Purnanand Elango, and Avishai Weiss. Deep reinforcement learning for station keeping on near rectilinear halo orbits. In *AAS/AIAA Astrodynamics Specialist Conference*, 2023. AAS 23-401.

[15] Victor Szebehely. *Theory of Orbits: The Restricted Problem of Three Bodies*. Academic Press, 1967.